APPENDIX A
PRE-SURVEY QUESTIONS

1) What is your username? (non-identifying unique name)
2) What is your email address or discord ID for prize notification? (optional)
3) Are you affiliated with the organizing institution as an employee? (required only for prize notification) (Yes/No)
4) Are you affiliated with the organizing institution as a student? (required only for prize notification) (Yes/No)
5) What is your US resident status? (required only for prize notification) (US citizen, permanent resident, nonresident alien, not in the United States)
6) What is your current occupation? (Developer, Technical Lead, Security Expert, Bug Hunter, Engineering major, Hobbyist, Other)
7) How many years have you been working with fuzzing tools and/or fuzz driver development? (<3 years, 3-5 years, 6-10 years, 10+ years, Other)
8) How do you rate your experience with writing fuzz drivers? (No experience, Novice, Experienced, Expert)
9) How do you rate your experience using OSS-Fuzz, Libfuzzer, and AFL? (No experience, Beginner, Intermediate, Advanced, Expert)
10) How familiar are you with reading and understanding a large program in the following languages? [C]
11) How familiar are you with writing a program from start to finish in the following languages? (1=Not familiar, 2=Slightly familiar, 3=Somewhat familiar, 4=Very familiar, 5=Extremely familiar) [C]
12) How familiar are you with modifying a large program in the following languages? (1=Not familiar, 2=Slightly familiar, 3=Somewhat familiar, 4=Very familiar, 5=Extremely familiar) [C]
13) How familiar are you with reading and understanding a large program in the following languages? [C++]
14) How familiar are you with writing a program from start to finish in the following languages? (1=Not familiar, 2=Slightly familiar, 3=Somewhat familiar, 4=Very familiar, 5=Extremely familiar) [C++]
15) How familiar are you with modifying a large program in the following languages? (1=Not familiar, 2=Slightly familiar, 3=Somewhat familiar, 4=Very familiar, 5=Extremely familiar) [C++]
16) Which country do you live in?
17) What type of software do you develop/test? (Desktop applications, web applications, mobile applications, embedded software, enterprise applications)
18) How many years of experience do you have with software development?
19) How did you gain your development skills? (University, On the job, Free online courses, Paid online courses, Self-taught, Other)
20) If you are a student, what university are you enrolled in, and what is your subject?
21) Do you have a university degree? (Yes/No)

APPENDIX B
POST-SURVEY QUESTIONS

1) What is your username? (non-identifying unique name)
2) Please describe how you accomplish the following activities: What FUZZ PLATFORM and OSS-Fuzz projects did you work on?
3) What functions in the project did you try before achieving success and why did you choose them? What fuzz driver files (ie: newfuzzer.cc) did you modify?
4) Creating/modifying fuzz driver code and debugging compilation issues
5) Compiling the project and using different sanitizers
6) Resolving fuzzing issues such as slow operations and low coverage
7) Running the tools to check for improved code coverage
8) How would you compare the usability of the FUZZ PLATFORM challenges compared to the OSS-Fuzz challenges?
9) Beyond compiling and checking for code coverage, what FUZZ PLATFORM features and tools did you find useful? ('LOC' coverage calculators, Fuzz Introspector, Corpus, Dictionary, Address sanitizer, Memory sanitizer, Undefined behavior sanitizer, Thread sanitizer, Other)
10) What actions did you take outside the platform? (for example: webpages, other tools)
11) Which features did you use?
12) Did you find any crashes when using the platform? (Yes/No)
13) How would you rate the usability of the FUZZ PLATFORM? (1-5 Likert scale)
14) How would you rate the training videos, links, and exercises? (1-5 Likert scale)
15) Different fuzzing environments lead to unstable/unreliable results. When these machines have different environments, the consistency of the fuzzing output can be affected or disrupted. Examples: different Python versions and different Linux builds. (F1) (Questions 15-29 displayed a 1-5 Likert scale of Always, Often, Sometimes, Rarely, Never, I don't know)
16) Incompatibility between the fuzzing environment and the project leads to build failures. Compiler updates and versioning, specifically, can break fuzzing builds and require complex manual intervention from the developers to fix the problem. (F2)
17) Bad fuzz drivers lead to fuzzing failure or inconsistent results. (F6)
18) Issues with fuzzing test code lead to crashes/build failures. (F7)
19) Bugs in the fuzzer lead to build failures or abnormal / inconsistent fuzzing behaviors. This includes fuzzers not being able to detect known positive cases, generating false positives, reporting the wrong type of failure, or simply crashing. (F8)
20) Incorrect use of the corpus leads to fuzzing failures or low coverage. (F9)
21) Issues with the build tools or external dependencies lead

to crash/build failures. For example, docker images have grown too big over time. (F10)

22) Issues in the corpus (such as unreadable or bad data) lead to build failures or unreliable results. (F11)
23) Developers have difficulties generating the correct inputs or targeting specific parts of the software. For example, a fuzz driver achieving only shallow coverage or being unable to pass strict pre-condition checks. (F15)
24) Developers have difficulties in setting up, building, or using the fuzzer. This includes breaking the build, not fuzzing the right fuzz target, causing the fuzzer to use too much memory, and many other issues. (F16)
25) Documentation on how to use the fuzzer is missing/ insufficient. For example, finding information on how to utilize optional features of a fuzzing tool. (F17)
26) Messages/information generated by fuzzers is missing / confusing/unhelpful. For example, crashing outputs are found but not reported. (F18)
27) Bad code design limits the ability to fuzz. This includes insufficient segmentation/separation of the target code, monolithic design, and assumptions about intended behavior, often requiring tests be done on the full system. (F20)
28) Developers have difficulties writing/understanding and debugging fuzzing code. (F21)
29) Developers have difficulty deciding which parts of the software to fuzz. (F22)
30) Have you encountered other challenges not mentioned in this survey?
31) Do you have any general suggestions for how the platform or the fuzzing process could be improved?